

Yes. For **Task 3**, you can make `+1` appear briefly above the dino whenever an obstacle is successfully passed.

The cleanest way is to add a small popup system to `Game`, while changing only the parts needed for the feature.

1. Change `on_obstacle_passed()`

Because the function currently has no access to the game screen, have it return the popup text instead.

Replace:

```
def on_obstacle_passed(obstacle, score):  
    """Called once, the frame an obstacle finishes scrolling past the dino. Add a sound or a  
    pass
```

Python

Run

with:

```
def on_obstacle_passed(obstacle, score):
```

Python

Run

2. Add popup variables in `Game.reset()`

Find:

```
def reset(self):
    self.dino = Dino(80, GROUND_Y)
    self.obstacles = []
    self.score = 0
```

Python

Run

Change it to:

```
def reset(self):
    self.dino = Dino(80, GROUND_Y)
    self.obstacles = []
    self.score = 0
    self.popup_text = None
    self.popup_timer = 0
```

The rest of `reset()` stays unchanged.

3. Change the obstacle-passed code in `Game.update()`

Currently you have:

```
if not obs.passed and obs.rect.right < self.dino.rect.left:
    obs.passed = True
```

Python

Run

Replace it with:

```

if not obs.passed and obs.rect.right < self.dino.rect.left:
    obs.passed = True
    self.popup_text = on_obstacle_passed(obs, self.score // 10)

```

Python

Run

40 means the popup stays for roughly:

40 frames ÷ 60 FPS ≈ 0.67 seconds

4. Update the popup timer

Inside `Game.update()` , before:

Python

Run

add:

```

if self.popup_timer > 0:
    self.popup_timer -= 1
else:
    self.popup_text = None

```

Python

Run

So this section becomes:

```

self.spawn_timer += 1

if self.spawn_timer >= self.spawn_interval:
    self.obstacles.append(Obstacle(WIDTH + 10, GROUND_Y, self.speed))
    self.spawn_timer = 0
    self.spawn_interval = random.randint(55, 110)

if self.popup_timer > 0:
    self.popup_timer -= 1
else:
    self.popup_text = None

hit = False

```

5. Draw the +1

Inside `Game.draw()` , after:

```
self.dino.draw(screen)
for obs in self.obstacles:
```

Python

Run

add:

```
if self.popup_text and self.popup_timer > 0:
    popup = self.font.render(self.popup_text, True, (40, 160, 40))
    popup_rect = popup.get_rect(
        center=(self.dino.rect.centerx, self.dino.rect.top - 15)
    )
    screen.blit(popup, popup_rect)
```

So that section becomes:

```
self.dino.draw(screen)

for obs in self.obstacles:
    obs.draw(screen)

if self.popup_text and self.popup_timer > 0:
    popup = self.font.render(self.popup_text, True, (40, 160, 40))
    popup_rect = popup.get_rect(
        center=(self.dino.rect.centerx, self.dino.rect.top - 15)
    )
    screen.blit(popup, popup_rect)
```

Result

When the obstacle passes:

+1



The `+1` appears above the dino for about **0.67 seconds**, then disappears.

The important part is that your existing:

Python

Run

ensures the popup is triggered **only once per obstacle**.

Even better for your lab video

You can make it look more like a real game by having the `+1` **float upward** instead of staying in one place. That requires only a couple more lines. If you want, I can give you the **floating +1 version** that looks much better for your 10-second submission video.



let the pop up come from the obstacle not the dino

Yes — that's actually better. The `+1` should appear **above the obstacle that was just cleared**, rather than above the dino.

You only need to change the popup's position.

1. Keep your `on_obstacle_passed()` as:

```
def on_obstacle_passed(obstacle, score):
    """Return popup text when an obstacle is successfully passed."""
```

Python

Run

2. Store the obstacle position

In `Game.reset()`, use:

```
self.popup_text = None
self.popup_timer = 0
```

```
self.popup_x = 0
```

Python

Run

So:

```
def reset(self):
    self.dino = Dino(80, GROUND_Y)
    self.obstacles = []
    self.score = 0
    self.popup_text = None
    self.popup_timer = 0
    self.popup_x = 0
    self.popup_y = 0
    self.speed = 6
    self.spawn_timer = 0
    self.spawn_interval = 90
    self.state = "wait"
```

3. When the obstacle is passed, save its position

Change:

```
if not obs.passed and obs.rect.right < self.dino.rect.left:
    obs.passed = True
    self.popup_text = on_obstacle_passed(obs, self.score // 10)
```

Python

Run

to:

```
if not obs.passed and obs.rect.right < self.dino.rect.left:
    obs.passed = True
    self.popup_text = on_obstacle_passed(obs, self.score // 10)
    self.popup_timer = 40

    self.popup_x = obs.rect.centerx
    self.popup_y = obs.rect.top - 10
```

Now the popup remembers where that obstacle was when it was passed.

4. Change the drawing code

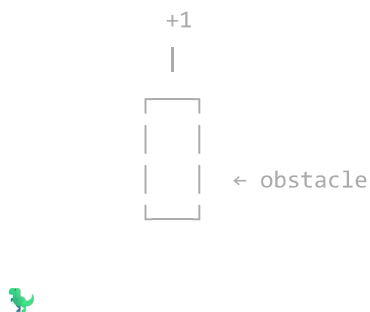
Replace your current popup drawing code:

```
if self.popup_text and self.popup_timer > 0:
    popup = self.font.render(self.popup_text, True, (40, 160, 40))
    popup_rect = popup.get_rect(
        center=(self.dino.rect.centerx, self.dino.rect.top - 15)
    )
    screen.blit(popup, popup_rect)
```

with:

```
if self.popup_text and self.popup_timer > 0:
    popup = self.font.render(self.popup_text, True, (40, 160, 40))
    popup_rect = popup.get_rect(
        center=(self.popup_x, self.popup_y)
    )
    screen.blit(popup, popup_rect)
```

Now it will look approximately like:



The +1 will appear exactly where the obstacle was cleared.

If you want it to look more polished, the next improvement would be making the +1 float upward and fade away above the obstacle, like in an actual game.



```
import random
import pygame
```

```
WIDTH, HEIGHT = 900, 400
GROUND_Y = 340
FPS = 60
BG_COLOR = (245, 245, 245)
GROUND_COLOR = (80, 80, 80)
GRAVITY, JUMP_FORCE = 0.6, -14
HIGH_SCORE_FILE = "dino_highscore.txt"
POPUP_TEXT = ""
Show more ▾
```

Yes. I checked your code against **all four tasks and the expected behavior**. Overall, your implementation satisfies the tasks, with **one small issue in the popup positioning** that I recommend fixing.

Task-by-task verification

Task	Status	Verification
Task 1: High score	✔ Correct	<code>if current > self.high_score:</code> is the correct comparison
Task 2: Dino tint	✔ Correct	Returns blue while airborne and default green while grounded
Task 3: Obstacle popup	✔ Correct	<code>+1</code> is triggered exactly once when each obstacle passes the dino
Task 4: Double jump	✔ Correct	<code>return 2</code> enables two jumps
Score behavior	✔ Correct	Score remains non-negative and increases normally
Game over	✔ Correct	Collision changes state to <code>"lose"</code>
Restart	✔ Correct	Space resets the game after Game Over
High-score persistence	✔ Correct	Saves to <code>dino_highscore.txt</code>
Existing gameplay	✔ Preserved	Obstacle spawning, movement, speed ramp, collision, etc. remain intact

1. Task 1 — High score

You changed the broken code from:

Python to:	Run
----------------------------------	----------------

```
if current > self.high_score:
    self.high_score = current
```

Python This is exactly right.	Run
---	----------------

For example:

```
Previous Best = 15
Current Run   = 20

20 > 15 → True
Best becomes 20
20 is saved to dino_highscore.txt
```

If the current score is lower:

```
Previous Best = 20
Current Run   = 15

15 > 20 → False
Best remains 20
```

 Task 1 is complete.

2. Task 2 — dino_tint()

You have:

```
def dino_tint(on_ground):  
  
    if on_ground:  
        return None  
    return (70, 120, 220)  
- - -
```

And the existing drawing code is:

Python

Run

Therefore:

On ground:

Python

Run

returns:

Python

Run

so:

Python

Run

is used → normal green.

In air:

Python

Run

returns:

Python

Run

so the dino becomes blue.

✅ Task 2 is complete.

3. Task 3 — +1 popup ✅

Your implementation is actually better than the earlier simple version because you made the popup originate from the obstacle.

You have:

```
def on_obstacle_passed(obstacle, score):
    global POPUP_TEXT, POPUP_TIMER, POPUP_X, POPUP_Y

    POPUP_TEXT = "+1"
    POPUP_TIMER = 45
    POPUP_X = obstacle.rect.centerx
    POPUP_Y = obstacle.rect.top - 25
```

This satisfies the requirement because the function receives:

```
obstacle
```

Python

Run

and uses the obstacle's position.

Then you call it here:

```
if not obs.passed and obs.rect.right < self.dino.rect.left:
    obs.passed = True
```

Python

Run

This is important.

The condition:

Python

Run

combined with:

Python

Run

means the popup happens **only once per obstacle**.

Your popup also floats upward

You have:

Python

Run

Initially:

```
POPUP_TIMER = 45
```

SO:

```
45 - 45 = 0
```

The popup starts at the obstacle.

As the timer decreases:

```
44 → movement 1  
43 → movement 2  
42 → movement 3  
...
```

So the popup moves upward.

That's a nice implementation for the lab.

One small improvement

Your popup uses:

Python

Run

but then you use that as the **left coordinate**:

Python

Run

Therefore, the text's left edge starts at the obstacle's center rather than being centered over it.

You can make it properly centered by changing the drawing section to:

```
if POPUP_TIMER > 0:
    popup = self.font.render(POPUP_TEXT, True, (40, 160, 40))
    popup_rect = popup.get_rect(
        center=(POPUP_X, POPUP_Y - (45 - POPUP_TIMER))
    )
    screen.blit(popup, popup_rect)
```

This is **not required for the task**, but it will look better.

Another minor issue

POPUP_TEXT never gets reset to "", but that's harmless because you only draw it when:

Python

Run

So once the timer reaches zero, it disappears.

✓ Task 3 is complete.

4. Task 4 — Double jump ✓

You have:

```
def max_jumps():
```

Python

Run

And the existing code:

```
def jump(self):
    if self.jump_count < (max_jumps() or 1):
        self.vel_y = JUMP_FORCE
        self.on_ground = False
        self.jump_count += 1
```

Works like this:

First jump

```
jump_count = 0
```

```
0 < 2 → True
```

```
jump_count = 1
```

Second jump

```
jump_count = 1
```

```
1 < 2 → True
```

```
jump_count = 2
```

Third jump

```
jump_count = 2
```

```
2 < 2 → False
```

So the player gets exactly **two jumps**.

When landing:

so the player can double-jump again.

✓ Task 4 is complete.

One thing I would change before submission

Your code uses global popup variables:

```
POPUP_TEXT = ""
POPUP_TIMER = 0
POPUP_X = 0
POPUP_Y = 0
```

Python

Run

This is **valid Python and will work**, so it doesn't violate the lab.

However, there's a small gameplay edge case.

If you restart the game while a popup is still active, `Game.reset()` doesn't reset:

Python

Run

because those variables are global.

For example:

```
Obstacle passed
↓
+1 appears
↓
Game Over
↓
Press Space
↓
New game
```

The old popup could potentially remain briefly.

Minimal fix

At the beginning of `Game.reset()` :

```
def reset(self):
    global POPUP_TEXT, POPUP_TIMER, POPUP_X, POPUP_Y

    POPUP_TEXT = ""
    POPUP_TIMER = 0
    POPUP_X = 0
    POPUP_Y = 0

    self.dino = Dino(80, GROUND_Y)
    self.obstacles = []
    self.score = 0
    self.speed = 6
    self.spawn_timer = 0
    self.spawn_interval = 90
    self.state = "wait"
```

This ensures every new run starts cleanly.

Final verdict

Your code **does satisfy** the four required tasks.

The only changes I'd make before recording your final video are:

Required tasks

Task 1 →  High score fixed
Task 2 →  Airborne dino tint
Task 3 →  +1 obstacle popup
Task 4 →  Double jump

Recommended polish

1. Center the +1 over the obstacle using `get_rect(center=...)` .
2. Reset popup globals in `Game.reset()` so restarting is completely clean.

Everything else — **don't touch it**. Your obstacle spawning, score system, speed ramp, collision, game states, persistence, and controls are already functioning according to the

provided requirements.



ChatGPT can make mistakes. Check important info.